

Prepare yourself as a best ui and artitacture base web designer that have all knowledge about privacy policy , terms & condition , remove froent end secretes , enforce https , cookies concent banners , mets title descriptions , social preview image , favicon , sitemap and robot.txt , image alt txt , image alt text , image compression , page load speed check , color constract fixes , mobile responsiviness , custome 404 page , broken link fixes , from validation , spam protection , analytatic setup , single clear cta.

so you have a good knowledge of this and use and required use this all things and remember to add this all things whenever you are creating the websites.

now understand we are working on a clothing breand websote that we create for gen-z brand name is cultraven in this I have working already in just like a template right now with that folder structure that we required I have already but if any file you wanted to add just add there.

now understand what I want I want gen-z tone into website with very professional website with best best ui like more premium breand but attractive for gen-z theme.

let see that theme code I exactly want here and how much I want to use that :
I need this color code here :

☐ Cream: **#EDE3CF**

☐ Navy: **#172554**

☐ Yellow: **#DAB205**

#F5F1E8 60% #172545 30% #C94227 10%

in this percentile you want to use this colors of use the current theme and % we have currently.

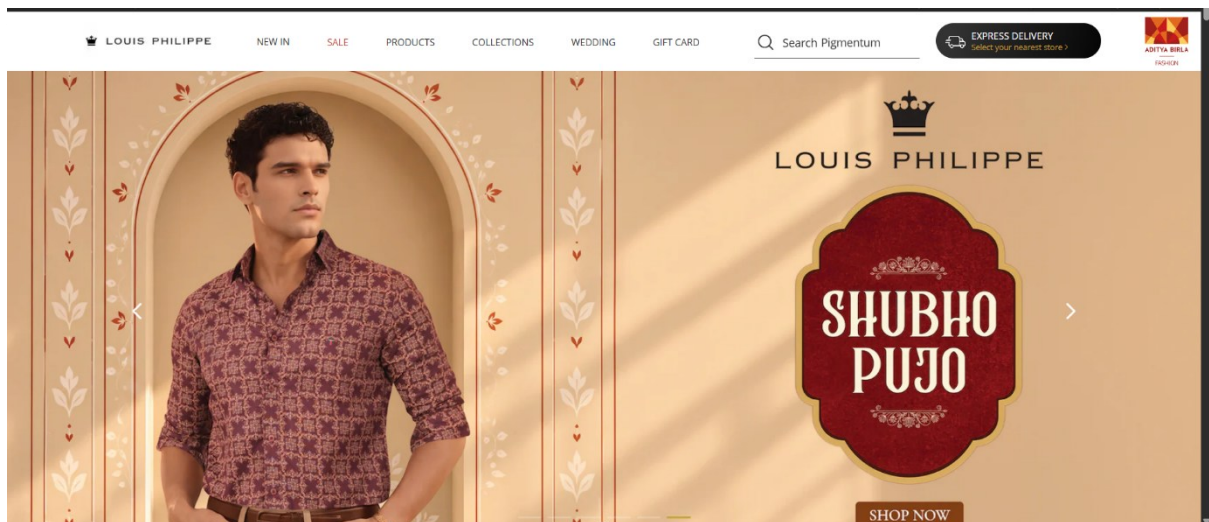
next what my basic requirements here let understand from here:

- 1: I am adding a whatsapp redirect popup into websote + a rag chatbot into that
- 2: rozarpay payment intrigation so added all pages of that
- 3: basic + advance features and apges for any e comer website we have always add into this too
- 4: fully functional admin panel by that we are abel to change anything content add product remove product everything that any e commerce websote required + hero section am able to change hero section image to show there offers banners and all
- 5: ui should be fully responsive and correct without any error all forms validation all typers to secure from all types of injection attacks
- 6: create a ui exact that samples images and details I provide to you
- 7: make sure you all apply all ui rules and functionality make sure everything has been working and prefect with same speed have currently
- 8: make sure this all updates you wanted to make into my current app in that folder structure I will have done already.
- 9: make sure entire ui + backend fully working and written in very optimized code also should be

clean and understandable code as per current folder structure make all apis and end points should be perfect and currently working on current versal deployment pipeline.
10: please use current theme that we have currently.

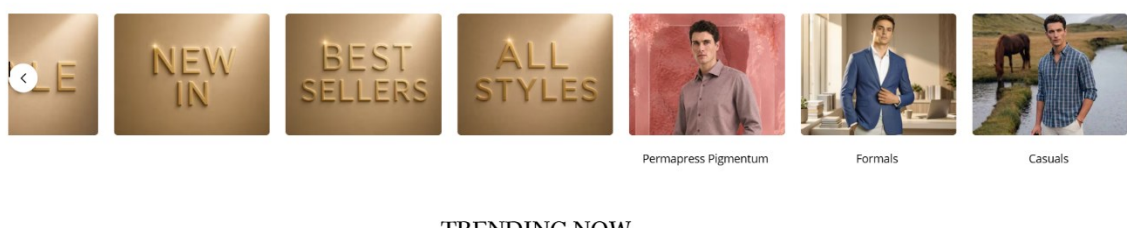
ui requirements :

I need all pages currently and all requirements properly all pages should be working and responsive and perfect:



Make sure in heading that options I provide that you are showing only and also this hero section images should be changeable by admin one or multiple I am able to upload and from admin fix the size what size is fix on hero section so just generate only that.

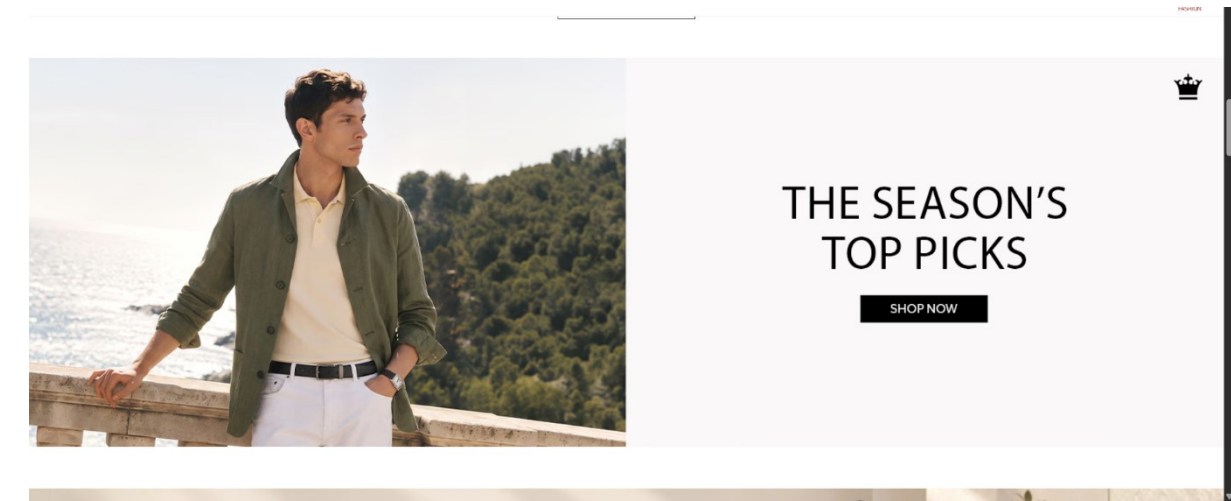
then after that this types of crawling options I also want and also this will be managed by admin if we upload that show in different pages as per entire website theme options only genz items. in this show all items we have like hoodies, jeans, etc for gen-z.



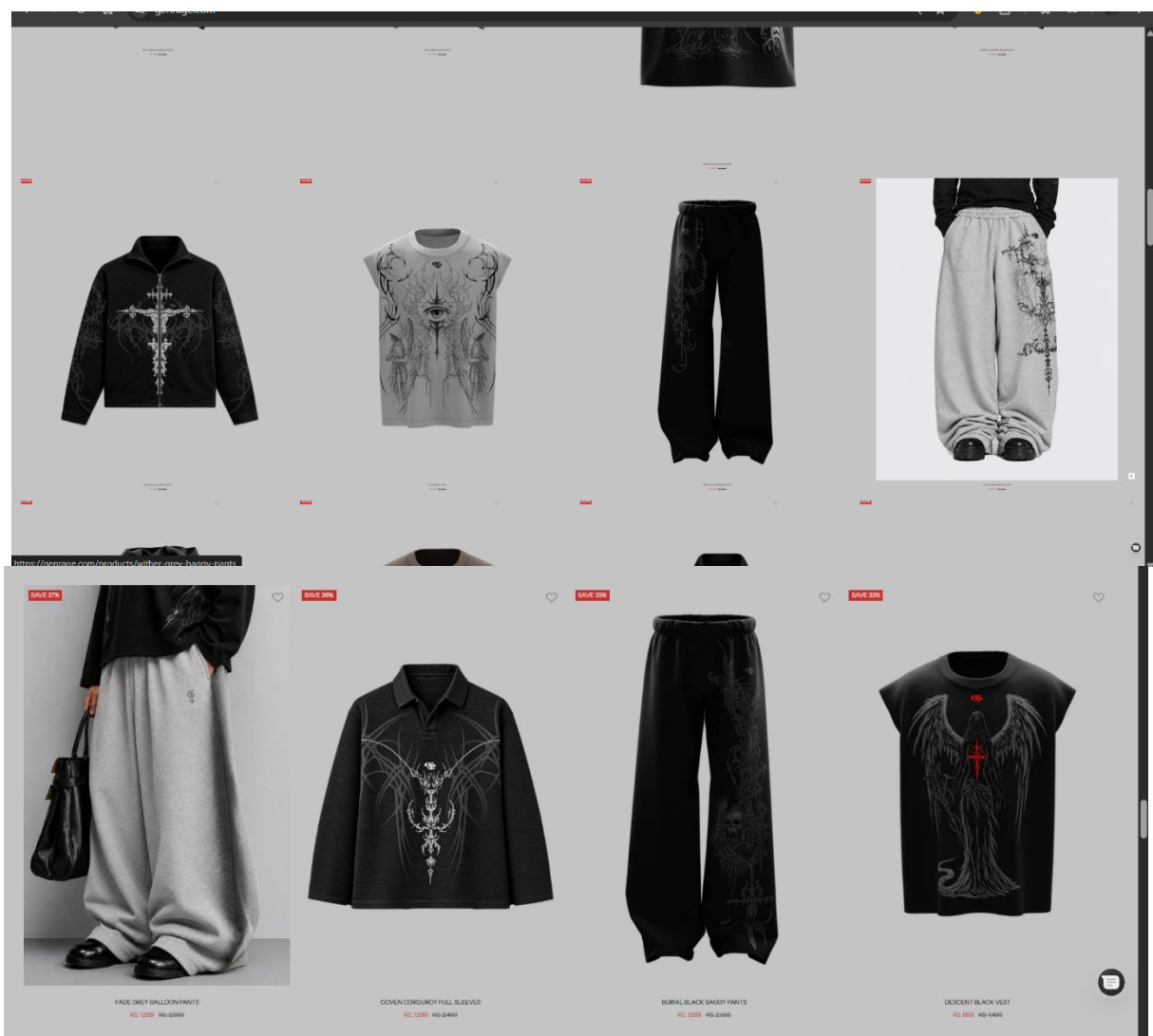
then show after this trending section where also we are able to add items from admin also here option single buy if another options are not adding also of added more clothes so show in new page everything handle and fully curd by admin panel.



After this add this banner but remember don't copy entirely we are creating for genz so create accordingly.



In center showing clothis like this so user easily click and get all details and add to cart and create proper all pages properly.this is also upload by ui that all big websote are work like mesho flipcart etc.



After that add this :



FREE SHIPPING

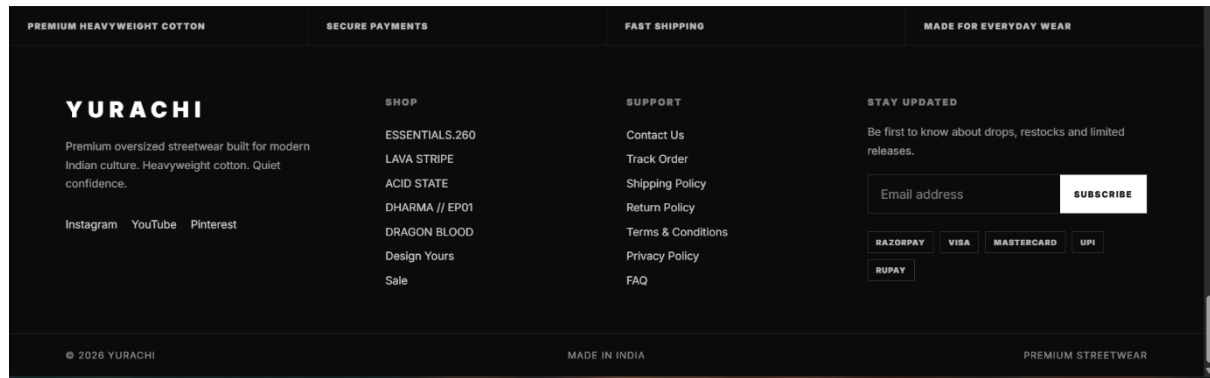


RETURN WITHIN 15 DAYS

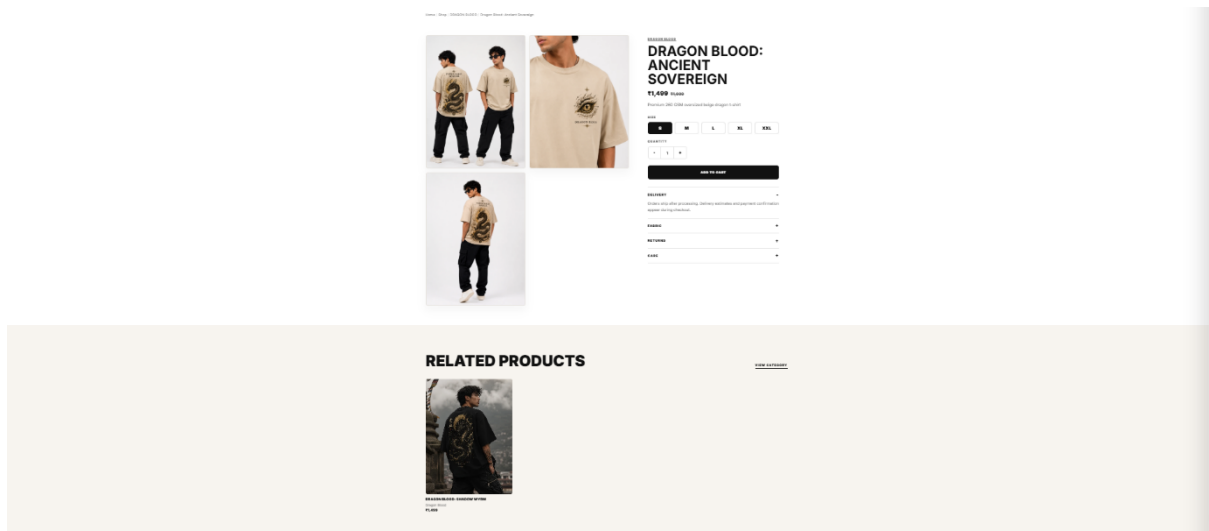


EXPRESS DELIVERY IN STORE MODE

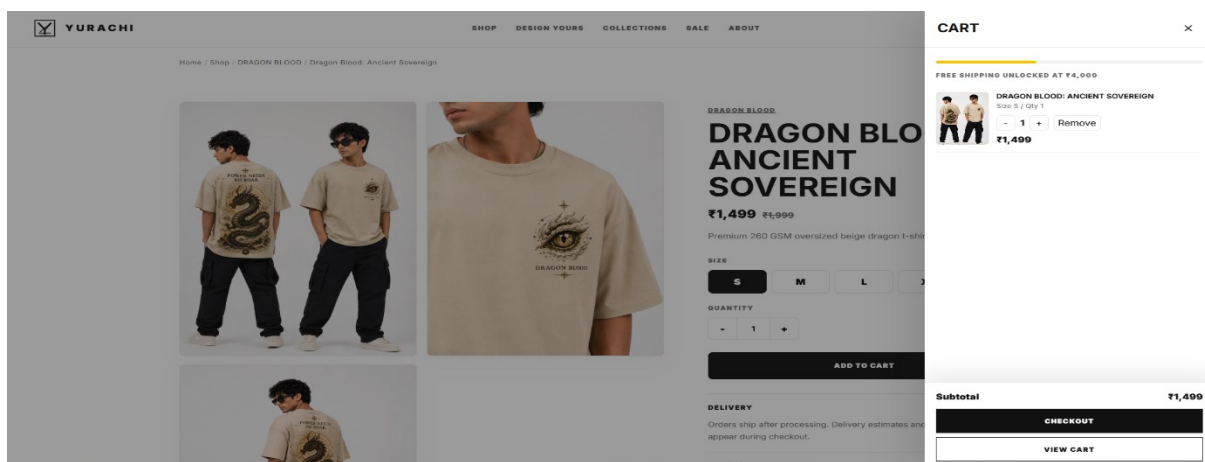
In footer I need this exact designe :



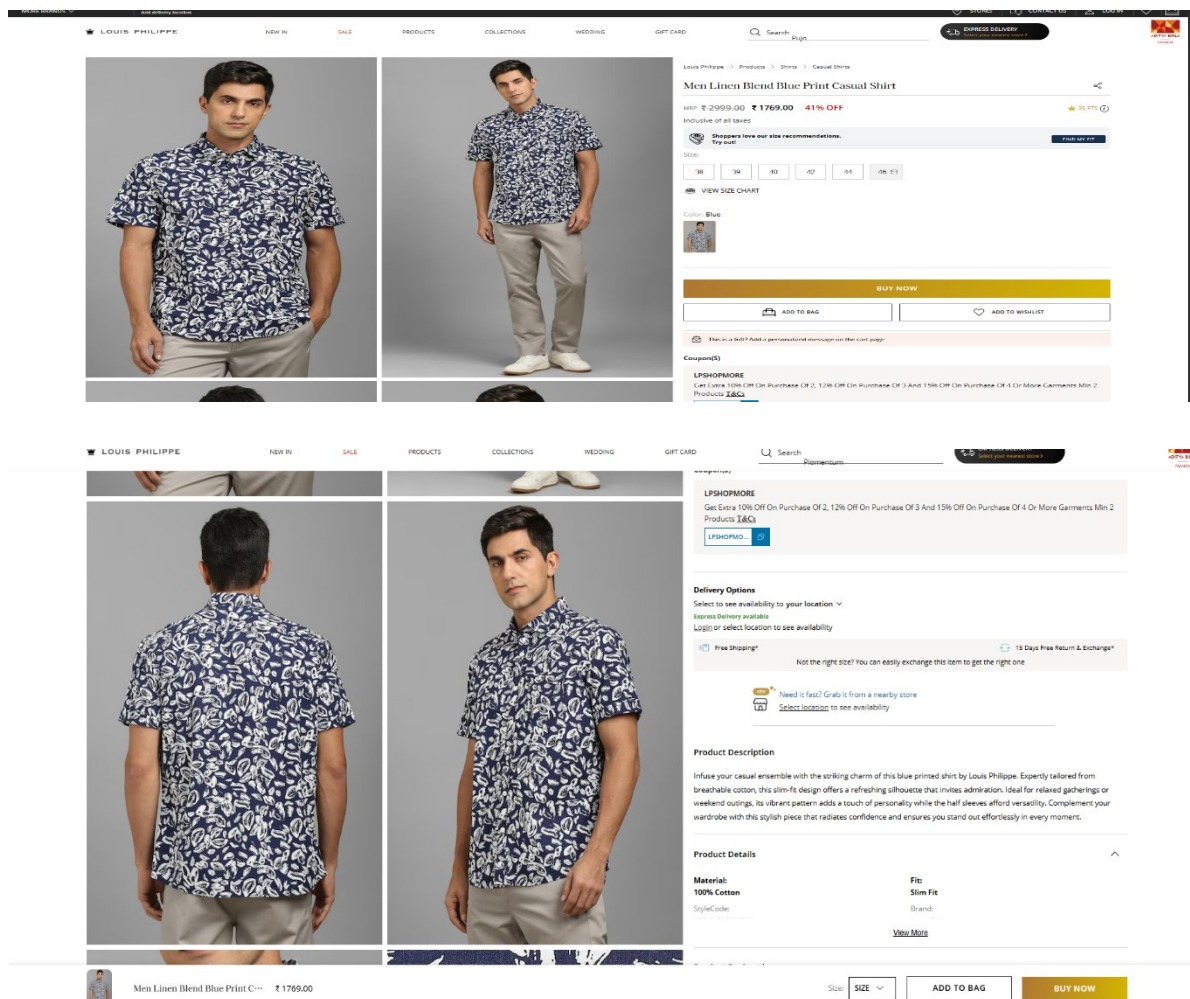
After clicok on any tshitt andy product show this exact screen :



When click add to card showing side baar and side page like this :

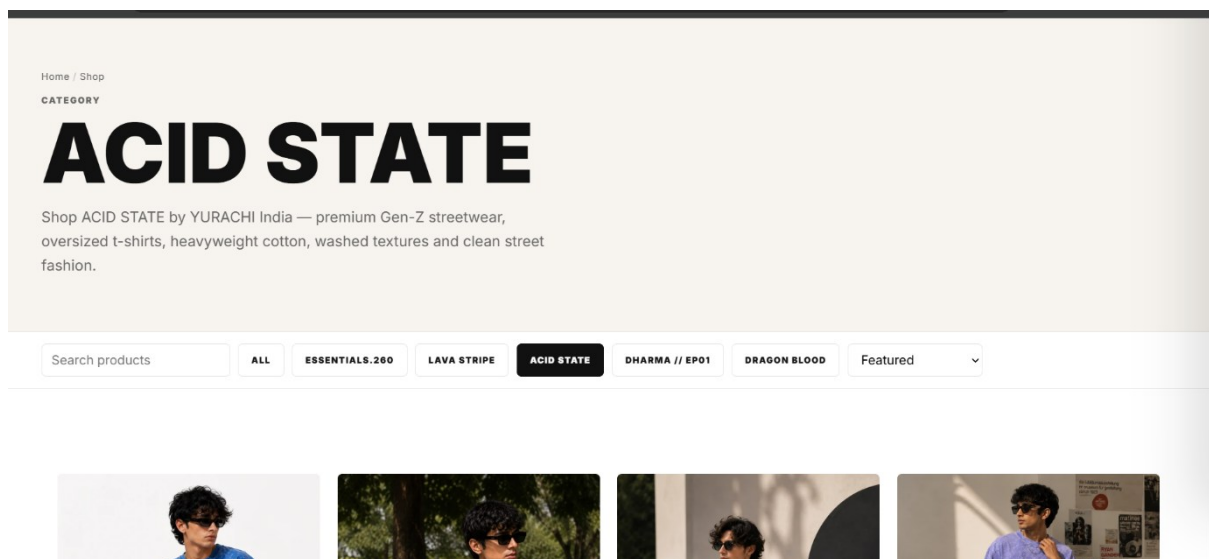


View cart option like this :this details also we are able to add by admin fully:

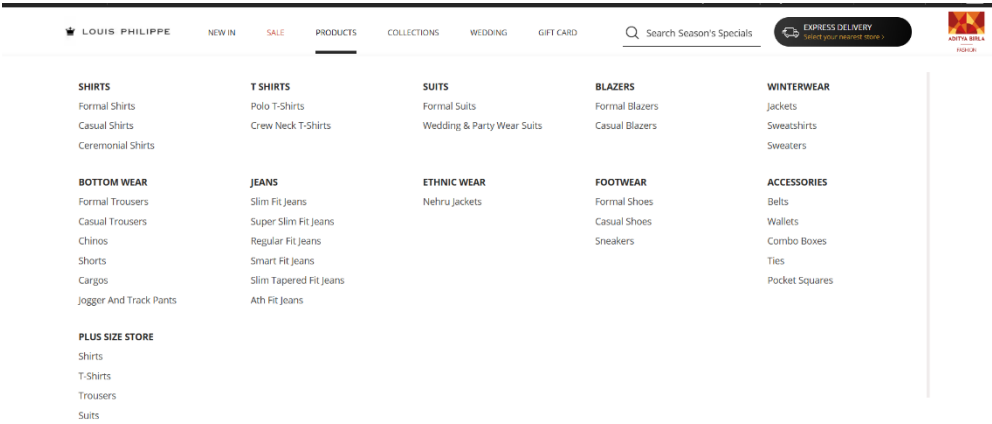
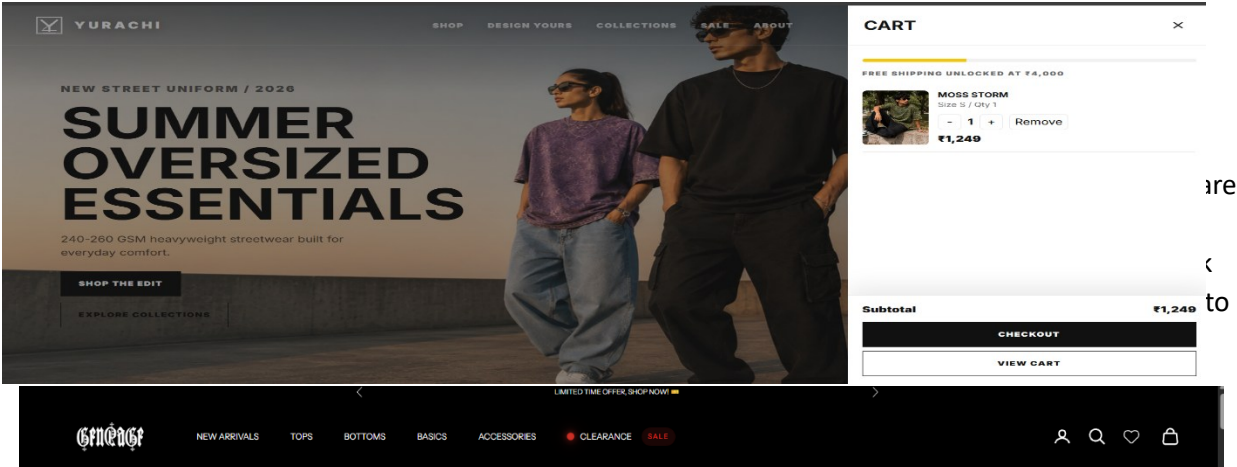


Note make sure all pages and all buttons code properly written without losing any single context and also make sure everything pages has been working properly.

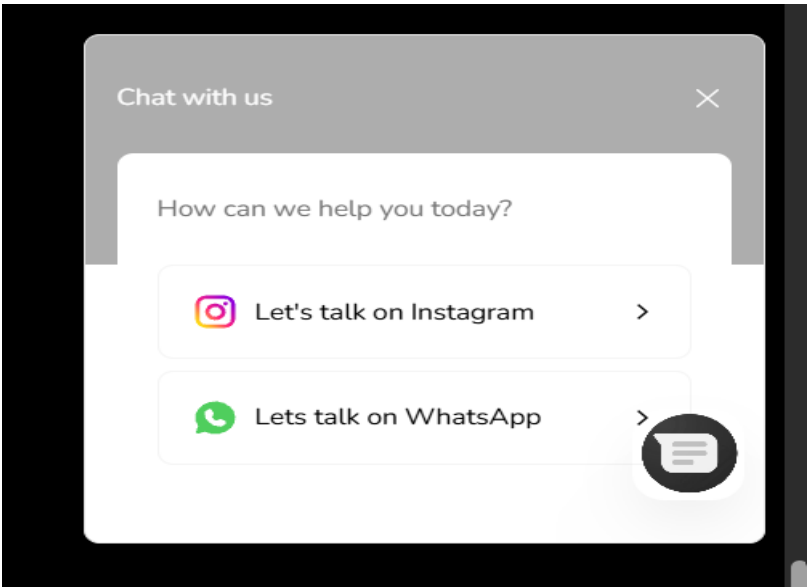
When a user click in an any other category so show like this with filters:



When click add to cart showing like this:



in whatsapp popup show option of whatsapp and Instagram and in chatrag show simple popup :



Rules:

IMPORTANT: Treat this as a REAL production website, not a demo, mockup, prototype, or purely visual concept.

You already have all the brand, design, layout, functionality, content, color, and business requirements I provided earlier. DO NOT replace or simplify those requirements. Build on top of them.

BRAND COLORS:

- Primary Cream: #EDE3CF
- Deep Navy: #172554
- Accent Yellow: #DAB205

The website must feel premium, modern, clean, intentional, and production-ready.

=====

1. PRODUCTION-FIRST DEVELOPMENT

=====

Before implementing anything, understand the entire existing codebase and architecture.

Do NOT blindly create new files or duplicate existing functionality.

First inspect:

- Project structure
- Framework and version
- Dependencies
- Existing components
- Existing routes
- API routes
- Database models
- Authentication

- Environment variables
- Middleware
- Existing utilities
- Image handling
- SEO configuration
- Build configuration
- Deployment configuration

Reuse existing architecture where appropriate.

Do not introduce unnecessary libraries.

Do not change working functionality unless required.

Do not create temporary hacks just to make the UI work.

Every implementation must be suitable for production.

=====

2. SECURITY — HIGH PRIORITY

=====

Perform a security review while building the website.

NEVER expose:

- API keys
- Database credentials
- JWT secrets
- NEXTAUTH_SECRET
- Payment secrets
- Private tokens

- OAuth secrets
- SMTP credentials
- Cloud credentials
- Admin credentials
- Internal API keys
- Private environment variables

Never hardcode secrets inside:

- React components
- JavaScript
- TypeScript
- HTML
- CSS
- API routes
- Git-tracked files

Use environment variables correctly.

Public environment variables must contain ONLY values that are intentionally safe for browser exposure.

Verify that:

- .env files are ignored by Git
- secrets are not committed
- secrets are not exposed in client bundles
- server-only environment variables stay server-side

If an environment variable is missing, fail safely and provide a useful server-side error rather than exposing sensitive information.

=====

3. AUTHENTICATION & AUTHORIZATION

=====

If authentication exists:

Implement secure:

- Login
- Logout
- Session handling
- Password handling
- Authorization
- Protected routes
- Admin routes
- Role-based access

Never trust the frontend for authorization.

Every sensitive API endpoint must independently verify:

1. Authentication
2. Authorization
3. User/session validity
4. Input validity

A hidden frontend button is NOT security.

Users must not be able to access admin APIs simply by manually entering URLs.

Prevent:

- Broken access control
- IDOR
- Privilege escalation

- Session manipulation
- Unauthorized API access

=====

4. INPUT VALIDATION

=====

Treat every user-controlled value as untrusted.

Validate input on BOTH:

Frontend:

- UX validation

Backend:

- Security validation

Validate:

- Forms
- Query parameters
- URL parameters
- Request bodies
- File uploads
- Search
- Contact forms
- Checkout data
- Admin inputs
- API payloads

Prevent:

- XSS

- SQL injection
- NoSQL injection
- Command injection
- Path traversal
- Malicious file uploads
- Prototype pollution
- Unsafe HTML injection

Use proper schema validation where appropriate.

Never trust client-side validation alone.

=====

5. API SECURITY

=====

Review every API endpoint.

Each endpoint must have:

- Proper HTTP method
- Authentication where required
- Authorization where required
- Input validation
- Proper error handling
- Rate limiting where appropriate
- Safe responses
- No sensitive information leakage

Never return:

- Password hashes
- Tokens

- Secrets
- Internal stack traces
- Database credentials
- Internal infrastructure details

Do not expose unnecessary database fields.

Return only the data required by the client.

=====

6. DATABASE SAFETY

=====

If a database is used:

Check:

- Connection handling
- Schema validation
- Indexes
- Unique constraints
- Null handling
- Data types
- Referential consistency
- Duplicate records
- Race conditions
- Query performance

Never construct database queries unsafely from raw user input.

Handle:

- Missing records

- Duplicate records
- Deleted records
- Invalid IDs
- Database connection failure
- Timeouts
- Partial failures

Do not expose raw database errors to users.

=====

7. PAYMENT SECURITY

=====

If payment functionality exists:

NEVER trust the frontend for payment verification.

Payment status must be verified server-side.

Implement:

- Secure payment creation
- Server-side verification
- Webhook verification
- Idempotency where required
- Duplicate payment protection
- Failed payment handling
- Cancelled payment handling
- Successful payment handling

Never store raw card information.

Never expose payment secret keys.

Never mark an order as paid solely because the browser says payment succeeded.

=====

8. ERROR HANDLING

=====

This is extremely important.

Find and fix hidden errors, not only visible errors.

Check for:

- Runtime errors
- Hydration errors
- React warnings
- Console errors
- Unhandled promises
- Failed API requests
- Race conditions
- Null/undefined errors
- Missing properties
- Broken imports
- Incorrect loading states
- Broken empty states
- Broken error states
- Network failures
- Timeout failures
- Database failures
- Third-party API failures

Do NOT use:

```
try {  
    ...  
} catch {  
    // ignore  
}
```

Do not silently swallow errors.

Every error should either:

- be handled correctly,
- be logged appropriately,
- or be surfaced to the correct user/system layer.

Never expose technical stack traces to normal users.

=====

9. LOADING / ERROR / EMPTY STATES

=====

Every asynchronous feature must have proper:

Loading state

Error state

Empty state

Success state

Do not leave users staring at a blank screen.

Examples:

API loading:

→ show appropriate loading UI

API failure:

→ show useful error message + retry option where appropriate

No data:

→ show meaningful empty state

Slow network:

→ maintain stable UI

=====

10. RESPONSIVE DESIGN

=====

The website must work properly on:

- iPhone
- Android
- Small mobile screens
- Large mobile screens
- Tablets
- Laptops
- Desktop
- Large monitors

Do not simply shrink desktop layouts.

Check:

- Navigation
- Hero
- Typography
- Images
- Buttons
- Forms
- Cards
- Product grids
- Footer
- Modals
- Tables
- Horizontal scrolling
- Touch targets

No accidental horizontal overflow.

No content should be clipped.

No overlapping elements.

No broken fixed/sticky elements.

Pay special attention to iOS Safari rendering and scrolling behavior.

Avoid unnecessary CSS hacks that can cause GPU/compositing problems.

=====

11. ACCESSIBILITY

=====

Implement proper accessibility.

Check:

- Semantic HTML
- Heading hierarchy
- Labels
- Form accessibility
- Keyboard navigation
- Focus states
- Button semantics
- Link semantics
- Alt text
- Color contrast
- Screen reader compatibility
- ARIA only when actually necessary

Do not use divs as buttons when a real button is appropriate.

All important interactions must work without a mouse.

=====

12. SEO

=====

Implement production-grade SEO.

Check:

- Page title
- Meta description
- Canonical URL
- Open Graph
- Twitter/X metadata

- Robots configuration
- Sitemap
- Structured data where appropriate
- Proper heading hierarchy
- Image alt attributes
- Internal linking
- Clean URLs

Avoid:

- Duplicate metadata
- Duplicate canonical URLs
- Incorrect canonical URLs
- Noindex accidentally applied to production
- Staging URLs appearing in production
- localhost URLs
- Old domain references
- Old brand metadata

IMPORTANT:

Search the entire project for:

- old domain
- localhost
- old brand name
- old company metadata
- old descriptions
- old URLs
- old images
- old schema
- old social links
- obsolete tracking IDs

Remove or replace stale production metadata.

=====

13. GOOGLE / SEARCH ENGINE SAFETY

=====

Ensure the website does NOT unintentionally use:

- Cloaking
- User-agent based content manipulation
- Hidden redirects
- Spam pages
- Hidden text
- Keyword stuffing
- Fake structured data
- Malicious redirects
- Suspicious scripts
- Unnecessary third-party scripts

The same legitimate page should be served consistently to normal users and search crawlers.

Do not create deceptive SEO behavior.

=====

14. PERFORMANCE

=====

Optimize for real-world performance.

Check:

- Core Web Vitals
- LCP
- CLS
- INP
- Image optimization
- Lazy loading
- Font loading
- JavaScript bundle size
- Code splitting
- Unnecessary dependencies
- Third-party scripts
- API response time
- Database queries

Do not sacrifice functionality merely to chase a Lighthouse score.

Avoid unnecessary:

- useEffect
- client components
- re-renders
- large libraries
- duplicated API calls

Use server-side rendering/server components where appropriate for the framework.

=====

15. IMAGES & MEDIA

=====

Ensure images:

- Have appropriate dimensions

- Are optimized
- Use modern formats where appropriate
- Do not cause layout shift
- Have meaningful alt text
- Do not contain broken URLs

Check every image source.

No broken image icons in production.

=====

16. FORMS

=====

Every form must handle:

- Empty input
- Invalid input
- Extremely long input
- Special characters
- Duplicate submissions
- Network failure
- Server failure
- Success
- Loading
- Validation errors

Prevent spam where appropriate.

Disable duplicate submission while a request is processing.

Never rely exclusively on frontend validation.

=====

17. THIRD-PARTY SERVICES

=====

Audit every external dependency:

- Analytics
- Meta Pixel
- Google services
- Payment gateway
- Email
- WhatsApp
- Maps
- CDN
- Authentication providers
- CMS
- APIs

If a third-party service fails, the entire website should not unnecessarily crash.

Use graceful degradation.

Do not block the main website because an optional analytics/tracking service fails.

=====

18. ANALYTICS & TRACKING

=====

Tracking must be implemented carefully.

Verify:

- Correct IDs
- Correct environment
- No duplicate events
- No accidental event loops
- No sensitive personal data sent to analytics
- Production/staging separation

If tracking is disabled or the ID is missing, the website should still function normally.

=====

19. SECURITY HEADERS

=====

Where supported by the hosting/framework, configure appropriate security headers such as:

- Content-Security-Policy
- Strict-Transport-Security
- X-Content-Type-Options
- Referrer-Policy
- Permissions-Policy
- Frame protection

Do not blindly add a CSP that breaks the website.

Test third-party scripts before enforcing restrictive policies.

=====

20. REDIRECTS & DOMAIN

=====

Ensure:

- HTTPS works
- HTTP redirects correctly
- www/non-www behavior is intentional
- No redirect loops
- No chained redirects
- Canonical domain is consistent
- Internal links use the correct production domain
- No localhost URLs remain

Check every important route directly.

=====

21. DEPLOYMENT

=====

The project must be deployment-ready.

Before considering the work complete, verify:

- Production build succeeds
- TypeScript checks pass
- Linting passes where configured
- No missing environment variables
- No broken imports
- No build warnings that indicate real problems
- No server/client boundary violations
- No deployment-only errors
- No environment-specific assumptions

Never say "done" simply because the development server starts.

=====

22. TESTING

=====

Before finalizing, perform a complete QA pass.

Test:

1. Homepage
2. Navigation
3. Every route
4. Every button
5. Every form
6. Every API
7. Authentication
8. Admin functionality
9. Database operations
10. Mobile layout
11. Tablet layout
12. Desktop layout
13. Error states
14. Empty states
15. Loading states
16. SEO metadata
17. Images
18. External integrations
19. Checkout/payment if applicable
20. 404 page

- 21. 500/error handling
- 22. Refreshing protected pages
- 23. Direct URL access
- 24. Back/forward browser navigation
- 25. Slow network behavior

=====

23. HIDDEN BUG AUDIT

=====

Do a dedicated "hidden bug audit" after implementation.

Look specifically for:

- Dead code
- Unused imports
- Unused state
- Race conditions
- Memory leaks
- Event listener leaks
- Duplicate requests
- Infinite render loops
- Infinite API loops
- Hydration mismatches
- Stale state
- Incorrect dependency arrays
- Missing cleanup
- Broken promises
- Incorrect async handling
- Null values
- Undefined values

- Missing database records
- Incorrect date/time handling
- Timezone bugs
- Floating-point money calculations
- Duplicate submissions
- Double-click bugs
- Refresh bugs
- Back-button bugs
- Browser-specific bugs
- Safari bugs
- Mobile viewport bugs

Fix root causes instead of hiding symptoms.

=====

24. CODE QUALITY

=====

Write maintainable code.

Avoid:

- Giant components
- Repeated code
- Magic numbers
- Hardcoded configuration
- Duplicate business logic
- Unnecessary abstraction
- Temporary hacks
- Commented-out dead code

Use:

- Clear naming
- Small reusable components
- Centralized configuration
- Proper types
- Proper error handling
- Consistent architecture

Do not over-engineer simple features.

=====

25. FINAL SECURITY SCAN

=====

Before finishing, search the entire project for potential secrets and sensitive data.

Look for patterns such as:

API_KEY

SECRET

PASSWORD

TOKEN

DATABASE_URL

MONGODB_URI

JWT_SECRET

NEXTAUTH_SECRET

PRIVATE_KEY

ACCESS_TOKEN

CLIENT_SECRET

Verify that none of these are accidentally exposed in client-side code or committed source files.

Also check Git history/configuration if available.

=====

26. FINAL SELF-REVIEW

=====

Do NOT stop after the first successful implementation.

After building:

STEP 1:

Review the entire implementation.

STEP 2:

Find potential bugs.

STEP 3:

Find security vulnerabilities.

STEP 4:

Find UX problems.

STEP 5:

Find responsive issues.

STEP 6:

Find SEO issues.

STEP 7:

Find performance problems.

STEP 8:

Find deployment problems.

STEP 9:

Fix them.

STEP 10:

Run the build/tests again.

Only then consider the feature production-ready.

=====

IMPORTANT RULE

=====

If you discover an existing bug or architectural problem while implementing the requested feature, DO NOT ignore it just because it was not explicitly mentioned.

Fix it if it is clearly within scope and safe to fix.

If fixing it could cause a breaking change, STOP and clearly explain:

- What is wrong
- Why it is dangerous
- What could break
- The recommended solution

Never silently introduce a risky workaround.

=====

FINAL RESPONSE FORMAT

=====

When the implementation is complete, report:

1. What was implemented
2. Security checks performed
3. Hidden bugs discovered and fixed
4. SEO checks
5. Performance checks
6. Responsive/device checks
7. Build/test results
8. Environment variables required
9. Any remaining warnings
10. Anything that still requires manual verification

Be honest.

Do not say "everything is perfect" unless it was actually verified.

If something could not be tested, explicitly say so.